

Web Application Security Assessment

Independent Penetration Testing Practice Report

Author: Abdelalim Saada

Context: Self-directed offensive security practice

Scope: Intentionally vulnerable web applications (DVWA, PortSwigger Academy labs, Metasploitable2)

Executive Summary

This report documents a self-directed security testing exercise performed against intentionally vulnerable web applications, as part of ongoing personal practice in offensive security. The objective was to apply hands-on testing methodology against common web vulnerabilities and document findings in a professional penetration testing report format. Testing concentrated on SQL Injection, Cross-Site Scripting (Reflected, Stored, DOM), Cross-Site Request Forgery (CSRF), and SSH brute-force attacks, using open-source labs (PortSwigger Academy, DVWA), industry-standard tools (Burp Suite, OWASP ZAP, SQLMap, Medusa), and a Kali Linux testing VM.

Methodology

Testing followed an iterative, non-destructive approach appropriate for learning labs and internal assessments:

1. Reconnaissance: identify target hosts, services and reachable web endpoints (Nmap, browser inspection).
2. Automated scanning: use Burp Suite and OWASP ZAP for passive and active discovery where applicable.
3. Manual verification: reproduce vulnerabilities manually and capture evidence (browser DevTools, Burp Proxy).
4. Exploitation (proof-of-concept only): demonstrate impact using controlled payloads and lab-provided exploit servers.
5. Reporting: document each finding with impact, proof-of-concept screenshots, and remediation guidance.

Findings

SQL Injection (SQLi)

Description: SQL Injection occurs when untrusted input is inserted into SQL queries without proper parameterization, allowing an attacker to manipulate queries and extract or modify backend database data.

Affected component: DVWA application

Proof of Concept / Evidence: Manual injection via URL parameters and SQLMap (see screenshots).

Impact: High — unauthorized data disclosure, authentication bypass, data modification depending on database privileges.

Remediation: Use prepared statements/parameterized queries, input validation, least privilege for DB accounts, and Web Application Firewall (WAF) rules.

Mapped OWASP Top 10 (2021): A03:2021 — Injection

Cross-Site Request Forgery (CSRF)

Description: CSRF enables attackers to make authenticated users perform unintended actions by leveraging existing sessions. The target lacked anti-CSRF tokens on sensitive state-changing endpoints (change email).

Affected component: Account settings — change email endpoint in PortSwigger Academy lab.

Proof of Concept / Evidence: Logged-in session before attack, and email changed in victim account after the exploit HTML was served (see screenshots).

Impact: Medium to High — attacker can change account details, enabling account takeover or persistence.

Remediation: Implement anti-CSRF tokens, require re-authentication for sensitive actions, use SameSite cookie attribute and validate Origin/Referer headers.

Mapped OWASP Top 10 (2021): A07:2021 — Identification and Authentication Failures

Cross-Site Scripting (XSS)

Reflected XSS

Description: Reflected XSS occurs when user input is immediately included in an HTTP response without proper encoding, leading to script execution in the victim's browser.

Affected component: Search parameter reflected in HTML (e.g., <h4> tag) in PortSwigger lab.

Proof of Concept / Evidence: *alert(1) popup and page rendering injected HTML (see screenshots).*

Impact: Medium — can steal session tokens, perform actions on behalf of users, or phish content.

Remediation: Output encode/escape user inputs depending on context, use Content Security Policy, validate input and use HTTP-only cookies for session tokens.

Mapped OWASP Top 10 (2021): A03:2021 — Injection

Stored XSS

Description: Stored XSS stores attacker-controlled input on the server (e.g., comments) which is later rendered to other users, enabling persistent script execution.

Affected component: Comment input fields on blog/article pages in PortSwigger stored XSS lab.

Proof of Concept / Evidence: *Injected JavaScript payload and confirmed lab solved (see screenshots).*

Impact: High — persistent XSS can compromise multiple users, steal credentials, or pivot to more severe attacks.

Remediation: Sanitize and escape stored content, use output encoding, implement input validation, and use libraries like DOMPurify for HTML contexts.

Mapped OWASP Top 10 (2021): A03:2021 — Injection

DOM-Based XSS

Description: DOM XSS occurs when client-side JavaScript copies untrusted data (e.g., location.search) into dangerous sinks (document.write, innerHTML) without sanitization.

Affected component: Client-side search handling using document.write / innerHTML in the lab.

Proof of Concept / Evidence: *Payload in URL and resulting script execution (see screenshots).*

Impact: Medium — depends on user interaction but can execute arbitrary scripts in victim's browser.

Remediation: Avoid unsafe DOM sinks, use safe DOM APIs, validate and sanitize any data used in the DOM, and apply CSP.

Mapped OWASP Top 10 (2021): A03:2021 — Injection

SSH Brute Force (Credential Attacks)

Description: Brute force attacks attempt multiple username/password combinations against an exposed SSH service to gain unauthorized access. The lab used default/weak credentials against Metasploitable2.

Affected component: SSH service on Metasploitable2 VM (port 22).

Proof of Concept / Evidence: *Medusa/Nmap outputs showing successful login with default credentials (see screenshots).*

Impact: High — unauthorized server access, lateral movement, data exfiltration.

Remediation: Disable password authentication, use key-based auth, enforce strong passwords, fail2ban/rate limiting, and monitor logs.

Mapped OWASP Top 10 (2021): A07:2021 — Identification and Authentication Failures

Conclusion

Overall, the tested lab environment demonstrated multiple common web vulnerabilities. Remediation should prioritize injection and authentication failures, and apply secure coding practices and defensive controls.

Tools & Platforms Used

Kali Linux · DVWA & PortSwigger Academy Labs · Metasploitable2 · Burp Suite · OWASP ZAP · SQLMap · Medusa · Nmap · Docker